

MANUAL TÉCNICO - FIUSAT Rating

Universidad de San Carlos de Guatemala
Facultad de Ingeniería - Escuela de Ciencias y Sistemas
Prácticas Iniciales 2026

Por Luis Zepeda

1. INTRODUCCIÓN

1.1 Propósito del Documento

Este manual técnico describe la implementación del servidor backend de la aplicación FIUSAT Rating, incluyendo la arquitectura, endpoints de la API REST, estructura de la base de datos, y procedimientos de instalación y configuración.

1.2 Alcance

Este documento está dirigido a:

Desarrolladores que necesitan entender la estructura del backend

Administradores que realizarán la instalación y mantenimiento

Evaluadores que verificarán el funcionamiento del sistema

1.3 Descripción del Sistema

FIUSAT Rating es una aplicación web que permite a los estudiantes de la Facultad de Ingeniería de la USAC:

Registrar y autenticar usuarios

Crear publicaciones sobre cursos y catedráticos

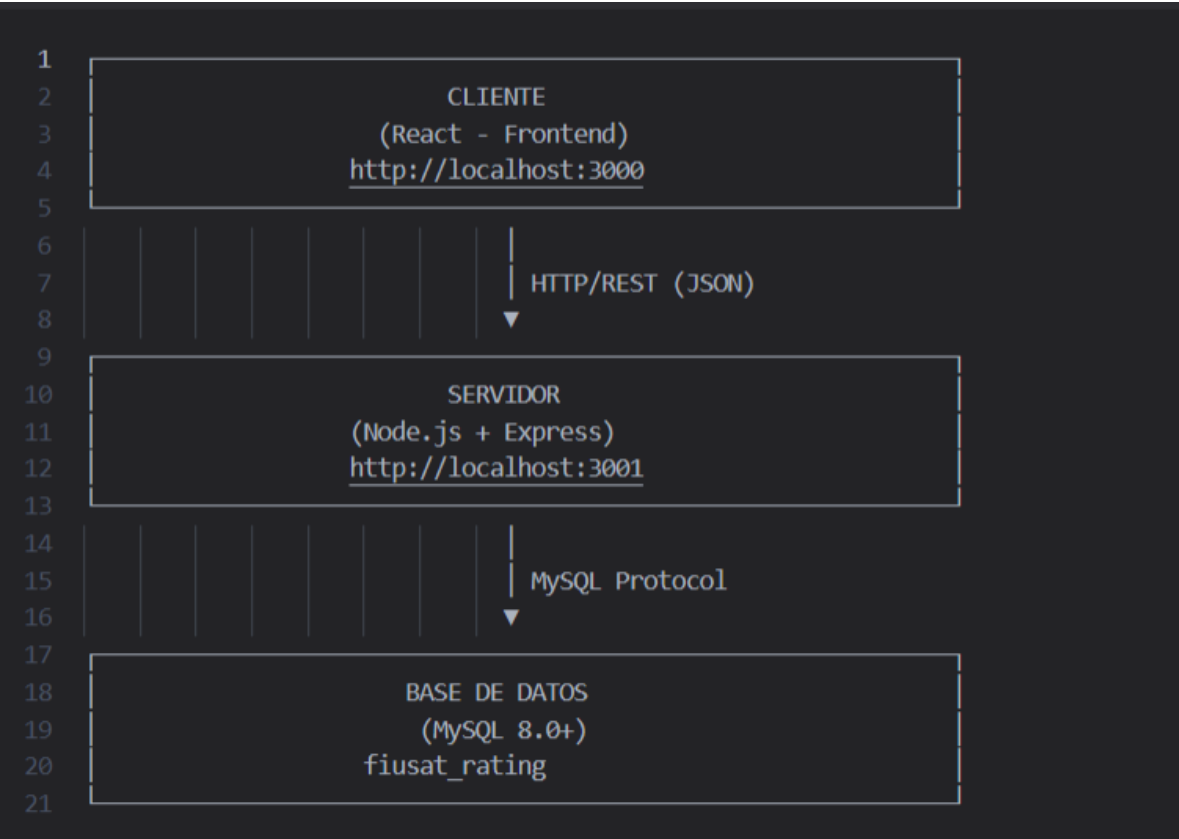
Comentar publicaciones de otros usuarios

Gestionar perfiles y cursos aprobados

Filtrar y buscar información

2. ARQUITECTURA DEL SISTEMA

2.1 Arquitectura Cliente-Servidor



2.2 Flujo de Datos

Paso	Descripción	Tecnología
1	Usuario interactúa con el frontend	React
2	Frontend envía petición HTTP	Axios
3	Backend recibe y valida la petición	Express
4	Backend consulta la base de datos	MySQL2
5	Base de datos retorna los datos	MySQL
6	Backend envía respuesta JSON	Express
7	Frontend muestra los datos al usuario	React

3. TECNOLOGÍAS UTILIZADAS

3.1 Backend

Tecnología	Versión	Propósito	↓
Node.js	18.x o superior	Entorno de ejecución	
Express	4.18.x	Framework web	
MySQL2	3.0.x	Conexión a base de datos	
Bcryptjs	2.4.x	Encriptación de contraseñas	
JSON Web Token	9.0.x	Autenticación	
Dotenv	16.0.x	Variables de entorno	
Cors	2.8.x	Permitir peticiones cruzadas	

3.2 Frontend

Tecnología	Versión	Propósito
React	18.x	Framework de UI
React Router DOM	6.x	Navegación entre rutas
Axios	1.x	Peticiones HTTP

3.3 Base de Datos

Tecnología	Versión	Propósito
MySQL Server	8.0+	Gestor de base de datos
MySQL Workbench	8.0+	Administración visual

3.4 Herramientas de Desarrollo

Herramienta	Propósito
Visual Studio Code	Editor de código
Postman	Pruebas de API
Git & GitHub	Control de versiones
npm	Gestor de paquetes

4. ESTRUCTURA DEL PROYECTO

4.1 Árbol de Directorios

fiusat_rating/

```
|
|
├── backend/
|   ├── node_modules/
|   ├── routes/
|   |   ├── auth.js      # Autenticación (login, registro, reset)
|   |   ├── usuarios.js  # Gestión de usuarios
|   |   ├── publicaciones.js # CRUD de publicaciones
|   |   ├── comentarios.js # CRUD de comentarios
|   |   ├── cursos.js    # CRUD de cursos
|   |   └── catedraticos.js # CRUD de catedráticos
|   ├── .env             # Variables de entorno
|   ├── db.js            # Conexión a MySQL
|   ├── index.js         # Punto de entrada del servidor
|   └── package.json      # Dependencias del proyecto
```

```
└─ package-lock.json    # Versiones exactas de dependencias
|
|
└─ frontend/
|   └─ node_modules/
|   └─ public/
|       └─ index.html
|   └─ src/
|       └─ components/
|           ├── Login.jsx
|           ├── Register.jsx
|           ├── Feed.jsx
|           ├── PublicacionCard.jsx
|           ├── CrearPublicacion.jsx
|           ├── Perfil.jsx
|           ├── CursosAprobados.jsx
|           ├── Navbar.jsx
|           ├── Login.css
|           ├── Register.css
|           ├── Feed.css
|           ├── PublicacionCard.css
|           ├── CrearPublicacion.css
|           ├── Perfil.css
|           ├── CursosAprobados.css
|           └─ Navbar.css
|       └─ services/
|           └─ api.js    # Configuración de Axios
```

```

| | └─ App.js      # Componente principal y rutas
| | └─ index.js    # Punto de entrada de React
| | └─ index.css   # Estilos globales
| └─ package.json
| └─ package-lock.json
|
└─ .gitignore      # Archivos a ignorar en Git
└─ README.md       # Documentación del repositorio
└─ database.sql    # Script de creación de BD

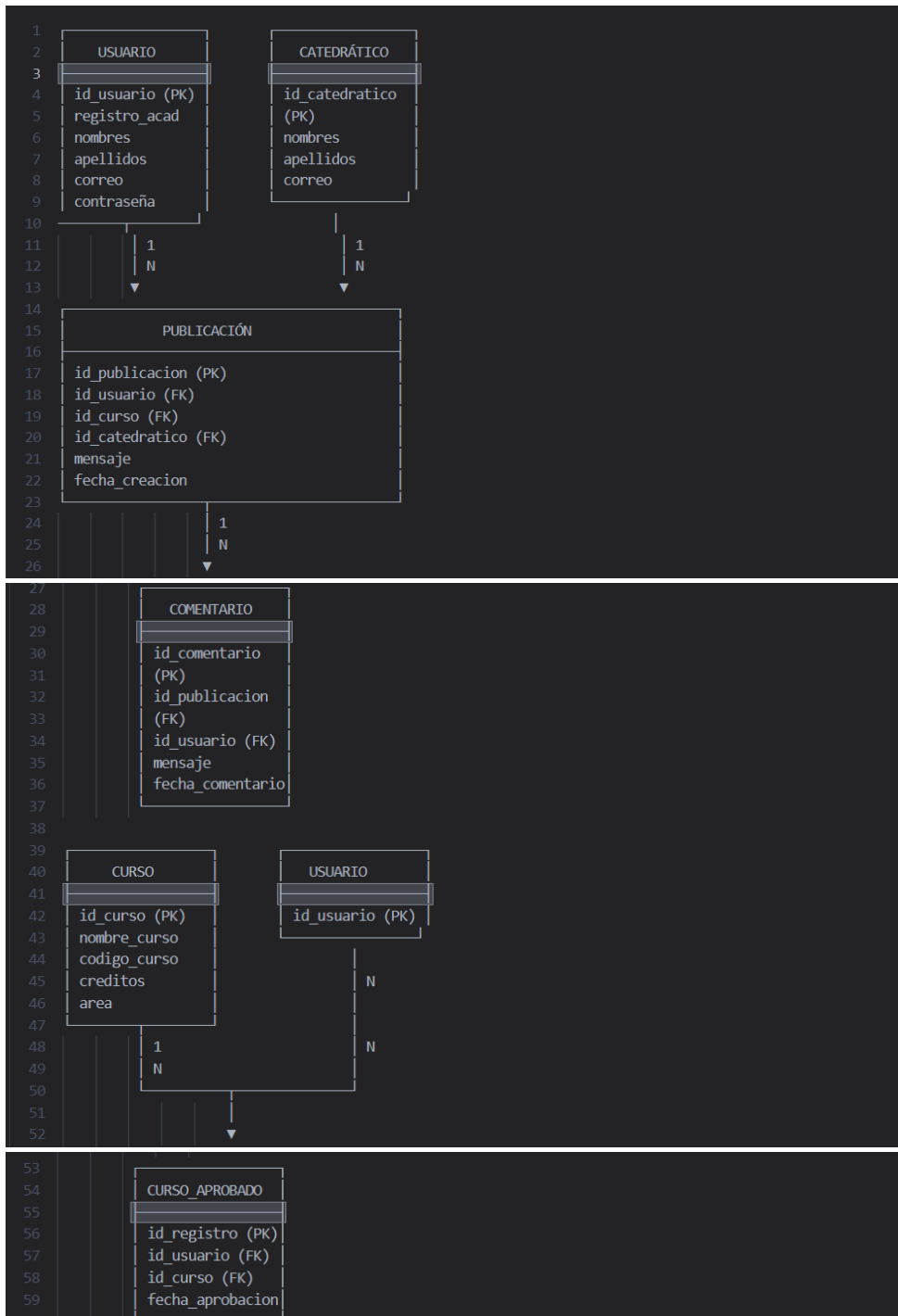
```

4.2 Descripción de Archivos Principales:

Archivo	Descripción
<code>backend/index.js</code>	Configura Express, middleware y monta las rutas
<code>backend/db.js</code>	Establece conexión con MySQL usando pool de conexiones
<code>backend/routes/auth.js</code>	Maneja registro, login y reset de contraseña
<code>backend/routes/usuarios.js</code>	CRUD de usuarios y cursos aprobados
<code>backend/routes/publicaciones.js</code>	CRUD de publicaciones con filtros
<code>backend/routes/comentarios.js</code>	CRUD de comentarios por publicación
<code>backend/routes/cursos.js</code>	Listado y gestión de cursos
<code>backend/routes/catedraticos.js</code>	Listado y gestión de catedráticos
<code>frontend/src/App.js</code>	Define las rutas de la aplicación React
<code>frontend/src/services/api.js</code>	Configura Axios con interceptores para tokens

5. BASE DE DATOS

5.1 Diagrama Entidad-Relación



5.2 Estructura de Tablas

Tabla: USUARIO

Columna	Tipo	Restricciones	Descripción
id_usuario	INT	PRIMARY KEY, AUTO_INCREMENT	Identificador único
registro_academico	VARCHAR(20)	NOT NULL, UNIQUE	Número de carnet
nombres	VARCHAR(100)	NOT NULL	Nombres del usuario
apellidos	VARCHAR(100)	NOT NULL	Apellidos del usuario
correo	VARCHAR(100)	NOT NULL, UNIQUE	Correo electrónico
contraseña	VARCHAR(255)	NOT NULL	Contraseña encriptada (bcrypt)
fecha_registro	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Fecha de creación

Tabla: CATEDRÁTICO

Columna	Tipo	Restricciones	Descripción	⬇
id_catedratico	INT	PRIMARY KEY, AUTO_INCREMENT	Identificador único	
nombres	VARCHAR(100)	NOT NULL	Nombres del catedrático	
apellidos	VARCHAR(100)	NOT NULL	Apellidos del catedrático	
correo	VARCHAR(100)	NULL	Correo institucional	

Tabla: CURSO

Columna	Tipo	Restricciones	Descripción	⬇
id_curso	INT	PRIMARY KEY, AUTO_INCREMENT	Identificador único	
nombre_curso	VARCHAR(100)	NOT NULL	Nombre del curso	
codigo_curso	VARCHAR(20)	NULL	Código del curso (ej: CC101)	
creditos	INT	NULL	Número de créditos	
area	VARCHAR(50)	DEFAULT 'Sistemas'	Área académica	

Tabla: PUBLICACIÓN

Columna	Tipo	Restricciones	Descripción	↓
id_publicacion	INT	PRIMARY KEY, AUTO_INCREMENT	Identificador único	
id_usuario	INT	FOREIGN KEY → usuario(id_usuario)	Autor de la publicación	
id_curso	INT	FOREIGN KEY → curso(id_curso), NULL	Curso relacionado	
id_catedratico	INT	FOREIGN KEY → catedratico(id_catedratico), NULL	Catedrático relacionado	
mensaje	TEXT	NOT NULL	Contenido de la publicación	
fecha_creacion	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Fecha de creación	

Tabla: COMENTARIO

Columna	Tipo	Restricciones	Descripción	↓
id_comentario	INT	PRIMARY KEY, AUTO_INCREMENT	Identificador único	
id_publicacion	INT	FOREIGN KEY → publicacion(id_publicacion)	Publicación padre	
id_usuario	INT	FOREIGN KEY → usuario(id_usuario)	Autor del comentario	
mensaje	TEXT	NOT NULL	Contenido del comentario	
fecha_comentario	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Fecha del comentario	

Tabla: CURSO_APROBADO

Columna	Tipo	Restricciones	Descripción	↓
id_registro	INT	PRIMARY KEY, AUTO_INCREMENT	Identificador único	
id_usuario	INT	FOREIGN KEY → usuario(id_usuario)	Usuario que aprobó	
id_curso	INT	FOREIGN KEY → curso(id_curso)	Curso aprobado	
fecha_aprobacion	DATE	NULL	Fecha de aprobación	
UNIQUE	(id_usuario, id_curso)	Evitar duplicados		

5.3 Script SQL de Creación

El archivo database.sql contiene el script completo para crear la base de datos. Ver sección 8.3.

6. API REST - ENDPOINTS

6.1 Convenciones

Elemento	Convención	Ejemplo
URL Base	<code>http://localhost:3001/api</code>	-
Método HTTP	GET, POST, PUT, DELETE	-
Formato de datos	JSON	<code>{"key": "value"}</code>
Códigos de estado	200, 201, 400, 401, 404, 500	-
Autenticación	Bearer Token en Header	<code>Authorization: Bearer <token></code>

6.2 Endpoints de Autenticación (/api/auth)

POST /api/auth/register

Registra un nuevo usuario en el sistema.

Parámetro	Tipo	Requerido	Descripción
registro_academico	String	Sí	8-9 dígitos numéricos
nombres	String	Sí	Nombres completos
apellidos	String	Sí	Apellidos completos
correo	String	Sí	Correo válido y único
contraseña	String	Sí	Mínimo 6 caracteres

Request:

json

```
1 POST http://localhost:3001/api/auth/register
2 Content-Type: application/json
3
4 {
5   "registro_academico": "202012345",
6   "nombres": "Juan Carlos",
7   "apellidos": "Pérez López",
8   "correo": "juan@test.com",
9   "contraseña": "123456"
10 }
```

Response (201 - Éxito):

json

```
1 {
2   "message": "Usuario registrado con éxito. Ahora puedes iniciar sesión."
3 }
```

Response (400 - Error):

json

```
1 {
2   "message": "El usuario ya existe. El correo o registro académico ya están registrados."
3 }
```

POST /api/auth/login

Autentica un usuario y devuelve un token JWT.

Parámetro	Tipo	Requerido	Descripción	
correo	String	Sí	Correo registrado	
contraseña	String	Sí	Contraseña del usuario	

Request:

```
json
1 POST http://localhost:3001/api/auth/login
2 Content-Type: application/json
3
4 {
5   "correo": "juan@test.com",
6   "contraseña": "123456"
7 }
```

Response (200 - Éxito):

```
json
1 {
2   "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
3   "user": {
4     "id": 1,
5     "nombres": "Juan Carlos",
6     "apellidos": "Pérez López",
7     "registro": "202012345",
8     "correo": "juan@test.com"
9   }
10 }
```

6.3 Endpoints de Usuarios (/api/usuarios)

GET /api/usuarios/:id

Obtiene la información de un usuario por ID.

Parámetro	Tipo	Ubicación	Descripción
id	INT	URL	ID del usuario

Request:

```
1 GET http://localhost:3001/api/usuarios/1
```

Response (200 - Éxito):

```
json
1 {
2   "id_usuario": 1,
3   "registro academico": "202012345",
4   "nombres": "Juan Carlos",
5   "apellidos": "Pérez López",
6   "correo": "juan@test.com"
7 }
```

GET /api/usuarios/registro/:registro

Busca un usuario por registro académico.

Parámetro	Tipo	Ubicación	Descripción	↓
registro	String	URL	Registro académico	

Request:

```
1 GET http://localhost:3001/api/usuarios/registro/202012345
```

6.4 Endpoints de Publicaciones (/api/publicaciones)

GET /api/publicaciones

Obtiene todas las publicaciones con filtros opcionales.

Parámetro	Tipo	Ubicación	Descripción	↓
id_curso	INT	Query	Filtrar por curso (opcional)	
id_catedratico	INT	Query	Filtrar por catedrático (opcional)	

Request:

```
1 GET http://localhost:3001/api/publicaciones
2 GET http://localhost:3001/api/publicaciones?id_curso=1
3 GET http://localhost:3001/api/publicaciones?id_catedratico=1
```

GET /api/publicaciones/:id

Obtiene una publicación específica por ID.

Parámetro	Tipo	Ubicación	Descripción	↓
id	INT	URL	ID de la publicación	

Request:

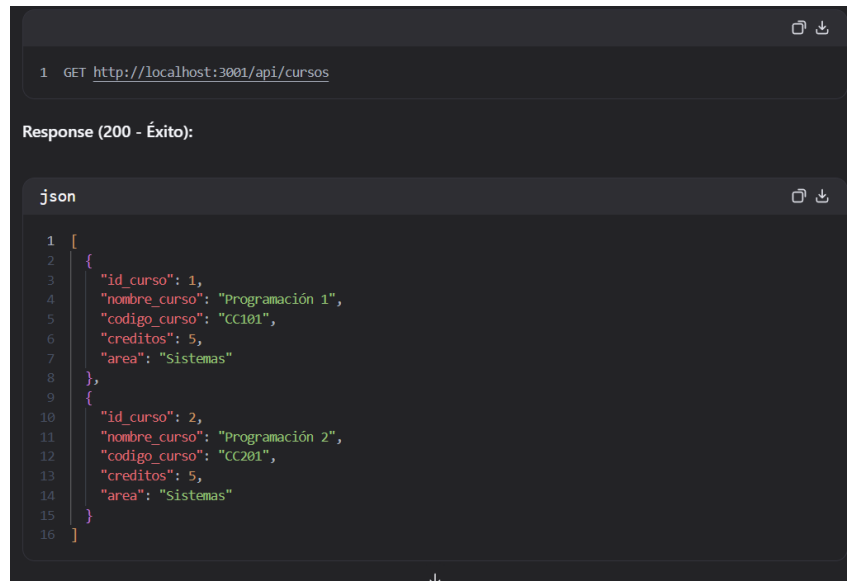
```
1 GET http://localhost:3001/api/publicaciones/1
```

6.6 Endpoints de Cursos (/api/cursos)

GET /api/cursos

Obtiene todos los cursos disponibles.

Request:



The screenshot shows a REST client interface. The top section displays a GET request to `http://localhost:3001/api/cursos`. Below this, the response is shown as a JSON array with two course objects. The response status is 200 - Éxito.

```
1 GET http://localhost:3001/api/cursos

Response (200 - Éxito):

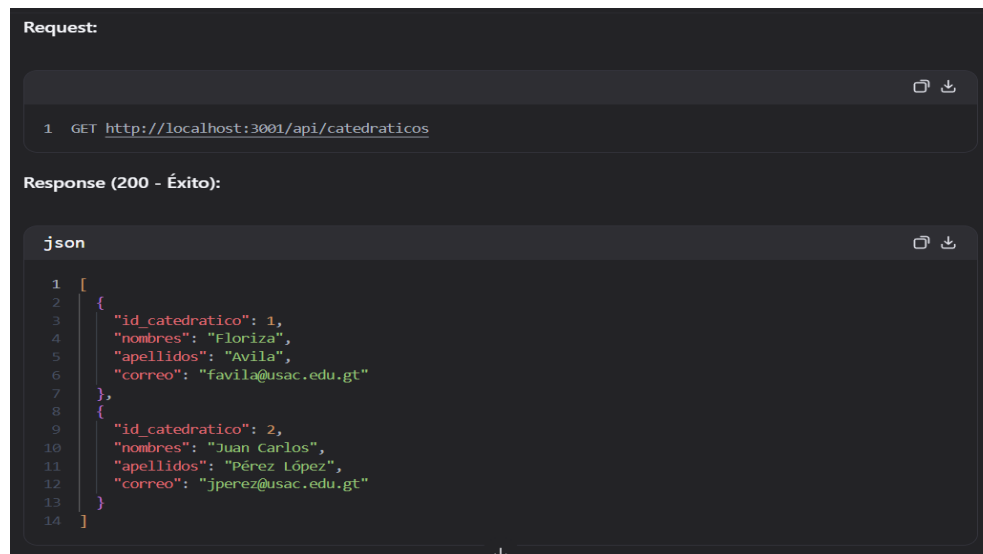
json

1 [
2   {
3     "id_curso": 1,
4     "nombre_curso": "Programación 1",
5     "codigo_curso": "CC101",
6     "creditos": 5,
7     "area": "Sistemas"
8   },
9   {
10    "id_curso": 2,
11    "nombre_curso": "Programación 2",
12    "codigo_curso": "CC201",
13    "creditos": 5,
14    "area": "Sistemas"
15  }
16 ]
```

6.7 Endpoints de Catedráticos (/api/catedraticos)

GET /api/catedraticos

Obtiene todos los catedráticos disponibles.



The screenshot shows a REST client interface. The top section displays a GET request to `http://localhost:3001/api/catedraticos`. Below this, the response is shown as a JSON array with two professor objects. The response status is 200 - Éxito.

```
Request:

1 GET http://localhost:3001/api/catedraticos

Response (200 - Éxito):

json

1 [
2   {
3     "id_catedratico": 1,
4     "nombres": "Floriza",
5     "apellidos": "Avila",
6     "correo": "favila@usac.edu.gt"
7   },
8   {
9     "id_catedratico": 2,
10    "nombres": "Juan Carlos",
11    "apellidos": "Pérez López",
12    "correo": "jperez@usac.edu.gt"
13  }
14 ]
```

7. AUTENTICACIÓN Y SEGURIDAD

7.1 Encriptación de Contraseñas

Las contraseñas se encriptan usando bcrypt con un salt de 8 rondas:

```
javascript
1  const bcrypt = require('bcryptjs');
2
3  // Encriptar contraseña
4  const hashedPassword = await bcrypt.hash(contraseña, 8);
5
6  // Verificar contraseña
7  const validPassword = await bcrypt.compare(contraseña, hashedPassword);
```

7.2 JWT (JSON Web Tokens)

Los tokens JWT se utilizan para autenticar las peticiones del frontend:

```
javascript
1  const jwt = require('jsonwebtoken');
2
3  // Generar token
4  const token = jwt.sign(
5    {
6      id: user.id_usuario,
7      registro: user.registro_academico,
8      nombres: user.nombres
9    },
10   process.env.JWT_SECRET || 'secreto_key_fiusat_2026',
11   { expiresIn: '24h' }
12 );
13
14 // Verificar token
15 const decoded = jwt.verify(token, process.env.JWT_SECRET);
```

7.3 Headers de Autenticación

Todas las peticiones protegidas deben incluir el token en el header

```
1  Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

7.4 Interceptores de Axios (Frontend)

El frontend usa interceptores para agregar automáticamente el token:

javascript

```
1  api.interceptors.request.use((config) => {
2    const token = localStorage.getItem('token');
3    if (token) {
4      config.headers.Authorization = `Bearer ${token}`;
5    }
6    return config;
7  });
8
9  api.interceptors.response.use(
10   (response) => response,
11   (error) => {
12     if (error.response?.status === 401) {
13       localStorage.removeItem('token');
14       localStorage.removeItem('user');
15       window.location.href = '/';
16     }
17     return Promise.reject(error);
18   }
19 );
```

8. INSTALACIÓN Y CONFIGURACIÓN

8.1 Requisitos Previos

Software	Versión Mínima	Enlace
Node.js	18.x	https://nodejs.org
npm	9.x	Incluido con Node.js
MySQL Server	8.0+	https://mysql.com
MySQL Workbench	8.0+	https://mysql.com/products/workbench
Git	2.x	https://git-scm.com
Visual Studio Code	Latest	https://code.visualstudio.com

8.2 Instalación del Backend

bash

```
1 # 1. Clonar o navegar al directorio del proyecto
2 cd backend
3
4 # 2. Instalar dependencias
5 npm install
6
7 # 3. Crear archivo .env
8 cp .env.example .env
9
10 # 4. Editar .env con las configuraciones de tu base de datos
11 # DB_HOST=localhost
12 # DB_USER=root
13 # DB_PASSWORD=tu_contraseña
14 # DB_NAME=fiusat_rating
15
16 # 5. Iniciar el servidor
17 npm start
18 # O con nodemon para auto-reiniciar
19 npm run dev
```

8.3 Instalación de la Base de Datos

sql

```
1 -- 1. Conectarse a MySQL
2 mysql -u root -p
3
4 -- 2. Crear la base de datos
5 CREATE DATABASE IF NOT EXISTS fiusat_rating;
6 USE fiusat_rating;
7
8 -- 3. Ejecutar el script de creación de tablas
9 source database.sql;
10
11 -- 4. Verificar que las tablas se crearon
12 SHOW TABLES;
13
14 -- 5. Insertar datos de prueba (opcional)
15 -- Ver archivo database.sql para los INSERT
```

8.4 Instalación del Frontend

bash

```
1 # 1. Navegar al directorio del frontend
2 cd frontend
3
4 # 2. Instalar dependencias
5 npm install
6
7 # 3. Iniciar el servidor de desarrollo
8 npm start
```

El frontend se abrirá automáticamente en <http://localhost:3000>

8.5 Variables de Entorno

Backend (.env)

env

```
1 # Configuración de la Base de Datos
2 DB_HOST=localhost
3 DB_USER=root
4 DB_PASSWORD=tu_contraseña_mysql
5 DB_NAME=fiusat_rating
6
7 # Configuración del Servidor
8 PORT=3001
9
10 # Configuración de JWT
11 JWT_SECRET=secreto_key_fiusat_2026_muy_seguro
```

Frontend

No requiere archivo .env, la URL del backend está configurada en `src/services/api.js`:

javascript

```
1 const api = axios.create({
2   |   baseURL: 'http://localhost:3001/api',
3   |   // ...
4   | });
```

8.6 Verificación de la Instalación

Backend

bash

```
1 # Ejecutar y verificar que el servidor inicia
2 node index.js
3
4 # Deberías ver:
5 # 🚀 Servidor corriendo en http://localhost:3001
6 # 🐼 API lista para recibir peticiones
```

Frontend

bash

```
1 # Ejecutar y verificar que React inicia
2 npm start
3
4 # Deberías ver en el navegador:
5 # http://localhost:3000
```

Base de Datos

sql

```
1 -- Verificar conexión y tablas
2 USE fiusat_rating;
3 SHOW TABLES;
4 SELECT * FROM usuario LIMIT 5;
```

9. PRUEBAS CON POSTMAN

9.1 Configuración de Postman

1. **Crear una Colección** llamada "FIUSAT Rating API"
2. **Crear Variables de Entorno:**

Variable	Valor
base_url	http://localhost:3001
token	(se llena después del login)

3. **Configurar Headers** en la colección:
 - Content-Type: application/json
 - Authorization: Bearer {{token}}

9.2 Flujo de Pruebas Recomendado

Orden	Endpoint	Método	Propósito	⬇
1	/api/auth/register	POST	Registrar usuario de prueba	
2	/api/auth/login	POST	Obtener token	
3	/api/cursos	GET	Ver cursos disponibles	
4	/api/catedraticos	GET	Ver catedráticos disponibles	
5	/api/publicaciones	POST	Crear publicación	
6	/api/publicaciones	GET	Ver publicaciones	
7	/api/comentarios	POST	Crear comentario	
8	/api/usuarios/:id	GET	Ver perfil	
9	/api/usuarios/:id/cursos	POST	Agregar curso aprobado	

10. Referencias

Tecnología	Enlace
Node.js	https://nodejs.org/docs
Express	https://expressjs.com
React	https://react.dev
MySQL	https://dev.mysql.com/doc
JWT	https://jwt.io
Axios	https://axios-http.com